

PricePoint: Machine Learning Laptop Valuation System

Comprehensive Project Report & Technical Documentation

Authors: Salman (@DeepSalman) & Contributors | Deployment: GitHub Pages (Serverless) | Target Currency: BDT (■)

1. Problem Description

In modern consumer electronics and secondary re-commerce markets, determining an accurate, fair market price for laptops is inherently difficult. Unlike commodities with standardized pricing, laptops feature a combinatorial explosion of specifications (CPU microarchitectures, RAM capacities, storage technologies, discrete GPU tiers, display panel quality, and physical condition).

Key challenges motivating this project include:

- **High Feature Non-Linearity:** Hardware pricing does not follow simple linear scaling. An additional 16 GB of RAM or a 4K OLED display adds significantly more monetary value to an Apple MacBook or Dell XPS workstation than to an entry-level student notebook.
- **Limitations of Heuristic Rules:** Simple rule-based or linear regression pricing formulas fail to model complex interactions (e.g., discrete GPUs paired with high-TGP gaming processors versus ultra-low-voltage ultrabooks).
- **Deployment Friction in Classical ML:** Conventional machine learning systems mandate always-on backend servers (FastAPI/Docker on AWS or Heroku), introducing latency, cold-starts, maintenance liabilities, and ongoing hosting expenses.

Project Goal: Develop a rigorous, benchmarked machine learning pricing engine and deploy it as a zero-latency, serverless in-browser web application accessible worldwide via GitHub Pages.

2. Methodology

The project followed an end-to-end data science lifecycle encompassing data exploration, string tokenization, robust feature engineering, supervised regression benchmarking, cross-validation, and deployment optimization.

2.1 Dataset & Feature Preprocessing

The dataset consists of **1,303 laptop entries** with extensive hardware attributes. The target variable is **price_bdt** (Bangladeshi Taka), converted from European retail benchmarks at ~128 BDT per EUR.

The feature space includes **6 categorical features** and **8 numerical features**:

- **Categorical:** Brand (19 manufacturers), Form Factor Type (6 categories), CPU Brand (6 tiers), Storage Drive Type (4 technologies), GPU Brand (Intel, Nvidia, AMD), and Operating System (6 families).
- **Numerical:** Screen Size (inches), Weight (kg), RAM (GB), Storage Capacity (GB), CPU Clock Speed (GHz), Resolution Area (total pixels), Touchscreen indicator (0/1), and IPS panel indicator (0/1).

Feature Engineering Highlights: Raw text columns required specialized parsing. Screen descriptions (e.g. *'IPS Panel Retina Display 2560x1600'*) were decomposed into binary IPS/Touchscreen indicators and total pixel area. Raw storage configurations (e.g. *'128GB SSD + 1TB HDD'*) were separated into primary capacity and drive technology.

2.2 Benchmarked Algorithms & Training Protocol

To identify the optimal model, six distinct machine learning algorithms were trained and evaluated:

- 1. **Linear Regression:** Standard Ordinary Least Squares baseline with one-hot encoded categories.
- 2. **Decision Tree Regressor:** Single CART decision tree capturing non-linear feature splits.
- 3. **Random Forest Regressor:** Bagging ensemble of 100 decorrelated decision trees.
- 4. **Support Vector Regressor (SVR):** Non-linear RBF kernel with standard scaling and log-transformed target (*TransformedTargetRegressor*).
- 5. **K-Nearest Neighbors (KNN):** Instance-based distance metric regression, tuned to optimal k=2 via cross-validation.
- 6. **Gradient Boosting Regressor:** Sequential boosting ensemble iteratively minimizing squared residuals.

Validation Protocol: The dataset was split into an 85% training set (1,107 samples) and a 15% holdout test set (196 samples) with fixed random seed 42. In addition, **5-fold cross-validation** was executed across the entire dataset to ensure generalization stability.

3. Results and Analysis

Models were scored using Mean Absolute Error (MAE in BDT), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R² Score).

Model Architecture	Test MAE (■)	Test RMSE (■)	5-Fold CV MAE	R ² Score	Role / Status
Gradient Boosting (120 Trees)	21,433	36,153	22,738	0.8534	Active Production
Random Forest (100 Trees)	23,022	39,431	22,695	0.8259	Ensemble Candidate
Support Vector Regressor (SVR)	22,027	38,574	23,600	0.8334	Scaled Kernel
K-Nearest Neighbors (KNN, k=2)	28,185	46,268	28,738	0.7594	Instance Matching
Decision Tree Regressor	30,412	55,876	29,513	0.6490	Single Tree Baseline
Linear Regression (One-Hot)	41,016	55,885	40,040	0.6489	Linear Baseline

3.1 Key Architectural Insights

- **Gradient Boosting Superiority:** Gradient Boosting achieved the highest R² score (0.8534) and lowest test MAE (■ 21,433). Because it fits sequential trees to residual errors, it excels at learning subtle spec combinations.
- **Failure of Linear Models:** Linear Regression exhibited the worst performance (MAE ■ 41,016). Linear combinations fail because high-end brands (e.g. Apple) have compounding pricing multipliers rather than constant addition.
- **Tree Bagging vs Boosting:** Random Forest performed admirably (R² = 0.8259), but Gradient Boosting's focused gradient descent in function space yielded tighter pricing boundaries on premium models.
- **Feature Scaling Necessity:** SVR and KNN only became competitive once features were normalized with StandardScaler and targets were log-transformed.

4. Implementation Details

The project is structured into two cleanly decoupled environments: an offline Python ML research suite and a standalone client-side web application.

4.1 Serverless Zero-Latency In-Browser Engine

To achieve instant client valuations without cloud hosting costs, the trained scikit-learn Gradient Boosting ensemble is exported into a compact JSON tree structure (**model_data.json**, ~79 KB).

A native ES-module JavaScript engine (**predictor.js**) executes client-side inference:

1. Categorical inputs are one-hot encoded in real-time matching the training feature vector.
2. The engine traverses 120 binary decision trees iteratively (left/right threshold evaluation).
3. Tree values are scaled by the learning rate (0.08) and summed with the initial base expectation.
4. Predictions evaluate in less than **1 millisecond** directly in the user's browser, completely offline-capable.

4.2 Domain-Aware Market Calibration

Real-world laptop pricing depends on factors beyond raw spec sheets. PricePoint combines raw ML model inference with secondary market parameters:

- **Release Era Depreciation:** Training data covers 2015–2020 listings. Modern 2024–2026 hardware receives a bounded generation factor (+15%), while legacy systems account for generational battery/silicon wear (-35%).
- **Cosmetic Condition Scaling:** Real used market pricing reflects condition grades (Brand New 100%, Excellent 95%, Good 90%, Fair 80%).

4.3 CLI Prediction & Testing Utility

A unified command-line tool (`ml/src/predict.py`) allows side-by-side evaluation across all 6 models:

```
$ python3 ml/src/predict.py --all
[Output] Linear: █141k | DecisionTree: █143k | SVR: █142k | KNN: █125k | GradientBoosting: █139k
```

5. Conclusion & Future Work

PricePoint demonstrates that sophisticated ensemble machine learning can be successfully packaged and deployed directly into static web environments without server dependencies. The resulting tool provides accurate, explainable, and zero-latency laptop market valuations.

Planned future improvements include expanding web scrapers for local marketplace price tracking and integrating deep learning embeddings for free-text model descriptions.